

```
// main function after the helper functions
async function yourMainFunctionName(bot) {
  // ...
}
'''
```

Prompt 5: Full system prompt for generating function descriptions. This is used when adding a new skill to the skill library. We give a one-shot example in the prompt.

You are a helpful assistant that writes a description of the given function written in Mineflayer javascript code.

- 1) Do not mention the function name.
- 2) Do not mention anything about 'bot.chat' or helper functions.
- 3) There might be some helper functions before the main function, but you only need to describe the main function.
- 4) Try to summarize the function in no more than 6 sentences.
- 5) Your response should be a single line of text.

For example, if the function is:

```
async function mineCobblestone(bot) {
  // Check if the wooden pickaxe is in the inventory, if not, craft
  one
  let woodenPickaxe = bot.inventory.findInventoryItem(mcData.
    itemsByName["wooden_pickaxe"].id);
  if (!woodenPickaxe) {
    bot.chat("Crafting a wooden pickaxe.");
    await craftWoodenPickaxe(bot);
    woodenPickaxe = bot.inventory.findInventoryItem(mcData.itemsByName
      ["wooden_pickaxe"].id);
  }

  // Equip the wooden pickaxe if it exists
  if (woodenPickaxe) {
    await bot.equip(woodenPickaxe, "hand");

    // Explore until we find a stone block
    await exploreUntil(bot, new Vec3(1, -1, 1), 60, () => {
      const stone = bot.findBlock({
        matching: mcData.blocksByName["stone"].id,
        maxDistance: 32
      });
      if (stone) {
        return true;
      }
    });

    // Mine 8 cobblestone blocks using the wooden pickaxe
    bot.chat("Found a stone block. Mining 8 cobblestone blocks.");
    await mineBlock(bot, "stone", 8);
    bot.chat("Successfully mined 8 cobblestone blocks.");

    // Save the event of mining 8 cobblestone
    bot.save("cobblestone_mined");
  } else {
    bot.chat("Failed to craft a wooden pickaxe. Cannot mine
      cobblestone.");
  }
}
```

The main function is 'mineCobblestone'.

Then you would write:

The function is about mining 8 cobblestones using a wooden pickaxe. First check if a wooden pickaxe is in the inventory. If not, craft one. If the wooden pickaxe is available, equip the wooden pickaxe in the hand. Next, explore the environment until finding a stone block. Once a stone block is found, mine a total of 8 cobblestone blocks using the wooden pickaxe.

A.4.3 Examples

Skill library example 1: craftWoodenPlanks.

```
async function craftWoodenPlanks(bot) {
  const logNames = ["oak_log", "birch_log", "spruce_log", "jungle_log",
    "acacia_log", "dark_oak_log", "mangrove_log"];
  const plankNames = ["oak_planks", "birch_planks", "spruce_planks", "jungle_planks", "acacia_planks", "dark_oak_planks", "mangrove_planks"];
  const logInInventory = logNames.find(logName => bot.inventory.count(mcData.itemsByName[logName].id) > 0);
  if (!logInInventory) {
    bot.chat("No wooden log in inventory. Mining a wooden log...");
    await mineWoodLog(bot);
  }
  const logIndex = logNames.indexOf(logInInventory);
  const plankName = plankNames[logIndex];
  bot.chat(`Crafting 4 ${plankName}...`);
  await craftItem(bot, plankName, 1);
  bot.chat(`4 ${plankName} crafted.`);
}
```

Skill library example 2: mineTenCobbledDeepslateBelowY0.

```
async function mineTenCobbledDeepslateBelowY0(bot) {
  // Equip the iron pickaxe
  const ironPickaxe = bot.inventory.findInventoryItem(mcData.itemsByName["iron_pickaxe"].id);
  await bot.equip(ironPickaxe, "hand");

  // Find cobbled_deepslate blocks below Y=0
  const cobbledDeepslateBlocks = await exploreUntil(bot, new Vec3(1, -1, 1), 60, () => {
    const cobbledDeepslate = bot.findBlock({
      matching: mcData.blocksByName["cobbled_deepslate"].id,
      maxDistance: 32,
      position: pos => pos.y < 0
    });
    return cobbledDeepslate;
  });

  // Mine 10 cobbled_deepslate blocks
  await mineBlock(bot, "cobbled_deepslate", 10);
  bot.chat("10 cobbled_deepslate mined below Y=0.");
}
```

Skill library example 3: smeltFiveRawIronV2.

```
async function findSuitablePosition(bot) {
  const offsets = [new Vec3(1, 0, 0), new Vec3(-1, 0, 0), new Vec3(0, 0, 1), new Vec3(0, 0, -1)];
  for (const offset of offsets) {
    const position = bot.entity.position.offset(offset.x, offset.y, offset.z);
    const block = bot.blockAt(position);
  }
}
```

```

    if (block.name === "air") {
        return position;
    }
}
return null;
}

async function smeltFiveRawIron(bot) {
    // Check if there is a furnace in the inventory
    const furnaceItem = bot.inventory.findInventoryItem(mcData.itemsByName.furnace.id);

    // If not, craft a furnace using the available cobblestone
    if (!furnaceItem) {
        await craftFurnace(bot);
    }

    // Find a suitable position to place the furnace
    const furnacePosition = await findSuitablePosition(bot);
    if (!furnacePosition) {
        bot.chat("Could not find a suitable position to place the furnace.");
        return;
    }

    // Place the furnace at the suitable position
    await placeItem(bot, "furnace", furnacePosition);

    // Smelt 5 raw iron using the available coal as fuel
    await smeltItem(bot, "raw_iron", "coal", 5);
    bot.chat("5 raw iron smelted.");
}

```

Skill library example 4: fillBucketWithWater.

```

async function fillBucketWithWater(bot) {
    // Find a water block nearby
    const waterBlock = await exploreUntil(bot, new Vec3(1, 0, 1), 60, ()
    => {
        const water = bot.findBlock({
            matching: mcData.blocksByName.water.id,
            maxDistance: 32
        });
        return water;
    });
    if (!waterBlock) {
        bot.chat("Could not find water.");
        return;
    }

    // Go to the water block
    const adjacentBlock = waterBlock.position.offset(0, 1, 0);
    await bot.pathfinder.goto(
        new GoalGetToBlock(adjacentBlock.x, adjacentBlock.y,
            adjacentBlock.z)
    );

    // Look at the water block
    await bot.lookAt(waterBlock.position, true);

    // Equip the bucket
    const bucket = bot.inventory.findInventoryItem(mcData.itemsByName.bucket.id);
    await bot.equip(bucket, "hand");
}

```

```

// Activate the bucket to collect water
await bot.activateItem();
bot.chat("Filled the bucket with water.");
}

```

Skill library example 5: catchFiveFishSafely.

```

async function catchFiveFishSafely(bot) {
  // Check if the bot has a fishing rod in its inventory
  let fishingRod = bot.inventory.findInventoryItem(mcData.itemsByName.fishing_rod.id);
  if (!fishingRod) {
    await craftFishingRod(bot);
    fishingRod = bot.inventory.findInventoryItem(mcData.itemsByName.fishing_rod.id);
  }

  // Find a nearby water block
  let waterBlock;
  while (!waterBlock) {
    waterBlock = await exploreUntil(bot, new Vec3(1, 0, 1), 60, () => {
      const foundWaterBlock = bot.findBlock({
        matching: mcData.blocksByName.water.id,
        maxDistance: 32
      });
      return foundWaterBlock;
    });
    if (!waterBlock) {
      bot.chat("No path to the water block. Trying to find another water block...");
    }
  }

  // Move to a block adjacent to the water block
  const adjacentBlock = waterBlock.position.offset(0, 1, 0);
  await bot.pathfinder.goto(new GoalBlock(adjacentBlock.x, adjacentBlock.y, adjacentBlock.z));

  // Look at the water block
  await bot.lookAt(waterBlock.position);

  // Equip the fishing rod
  await bot.equip(fishingRod, "hand");

  // Fish in the water 5 times
  for (let i = 0; i < 5; i++) {
    try {
      await bot.fish();
      bot.chat(`Fish ${i + 1} caught.`);
    } catch (error) {
      if (error.message === "Fishing cancelled") {
        bot.chat("Fishing was cancelled. Trying again...");
        i--; // Retry the same iteration
      } else {
        throw error;
      }
    }
  }
}

```

A.5 Self-Verification

A.5.1 Components in the Prompt

The input prompt to GPT-4 consists of the following components:

- (1) The agent’s state: We exclude other blocks that are recently seen and nearby entities from the agent’s state since they are not useful for assessing the task’s completeness. See Sec. A.3.1 for each element of the agent’s state;
- (2) Task proposed by the automatic curriculum;
- (3) Task context: We prompt GPT-3.5 to ask for general suggestions about how to solve the task. In practice, this part is handled by the automatic curriculum since it has a systematic mechanism for question-answering (Sec. A.3.2);
- (4) Chain-of-thought prompting [46] in response: We request GPT-4 to initially reason about the task’s success or failure, then output a boolean variable indicating the task’s outcome, and finally provide a critique to the agent if the task fails.
- (5) Few-shot examples for in-context learning [36–38].

A.5.2 Full Prompt

Prompt 6: Full system prompt for self-verification.

```
You are an assistant that assesses my progress of playing Minecraft
and provides useful guidance.

You are required to evaluate if I have met the task requirements.
Exceeding the task requirements is also considered a success while
failing to meet them requires you to provide critique to help me
improve.

I will give you the following information:

Biome: The biome after the task execution.
Time: The current time.
Nearby blocks: The surrounding blocks. These blocks are not collected
yet. However, this is useful for some placing or planting tasks.
Health: My current health.
Hunger: My current hunger level. For eating task, if my hunger level
is 20.0, then I successfully ate the food.
Position: My current position.
Equipment: My final equipment. For crafting tasks, I sometimes equip
the crafted item.
Inventory (xx/36): My final inventory. For mining and smelting tasks,
you only need to check inventory.
Chests: If the task requires me to place items in a chest, you can
find chest information here.
Task: The objective I need to accomplish.
Context: The context of the task.

You should only respond in JSON format as described below:
{
  "reasoning": "reasoning",
  "success": boolean,
  "critique": "critique",
}
Ensure the response can be parsed by Python 'json.loads', e.g.: no
trailing commas, no single quotes, etc.

Here are some examples:
INPUT:
Inventory (2/36): {'oak_log':2, 'spruce_log':2}
```